

DAY 10: PYTHON OBJECT-ORIENTED PROGRAMMING

60-Day Data Science Learning Journey | Muhammad Ahmad

Learning path: Classes → Objects → Attributes → `__init__` → `self` → Methods → Encapsulation → Inheritance → Polymorphism → Class/Static Methods → Composition → Data Science Project

1. What is OOP?

Object-Oriented Programming organizes software around objects. An object combines data and behavior. A class is the blueprint used to create objects.

```
class Student:
    pass

student1 = Student()
student2 = Student()
```

2. Classes and Objects

A class describes what an object has and can do. An object is a real instance of that class.

```
class Student:
    def __init__(self, name, cgpa):
        self.name = name
        self.cgpa = cgpa

ahmad = Student("Ahmad", 3.23)
ali = Student("Ali", 3.10)

print(ahmad.name, ahmad.cgpa)
print(ali.name, ali.cgpa)
```

3. Attributes, `__init__`, and `self`

Attributes are values stored in an object. `__init__` runs automatically when an object is created. `self` refers to the current object.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

student = Student("Ahmad", [80, 75, 90, 85])

print(student.name)
print(student.marks)
```

4. Methods

A method is a function inside a class. It can use the object's attributes through `self`.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average_marks(self):
        return sum(self.marks) / len(self.marks)

student = Student("Ahmad", [80, 75, 90, 85])
print(student.average_marks())
```

5. Updating Data

```
class Student:
    def __init__(self, name, cgpa):
        self.name = name
        self.cgpa = cgpa

    def update_cgpa(self, new_cgpa):
        self.cgpa = new_cgpa

student = Student("Ahmad", 3.23)
student.update_cgpa(3.40)
```

```
print(student.cgpa)
```

6. Encapsulation

Encapsulation keeps related data and behavior together and lets us control how data is changed.

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def get_balance(self):
        return self._balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())
```

`_balance` is a Python naming convention for internal use. It is not true private access control.

7. Inheritance

Inheritance lets a child class reuse or extend a parent class.

```
class Person:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print("Name:", self.name)

class Student(Person):
    def show_role(self):
        print("Role: Student")

student = Student("Ahmad")
student.show_name()
student.show_role()
```

8. super()

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, cgpa):
        super().__init__(name)
        self.cgpa = cgpa

student = Student("Ahmad", 3.23)
print(student.name)
print(student.cgpa)
```

9. Method Overriding

```
class Person:
    def role(self):
        print("I am a person")

class Student(Person):
    def role(self):
        print("I am a student")

Person().role()
Student().role()
```

10. Polymorphism

Polymorphism allows the same method name to behave differently for different object types.

```
class Student:
    def describe(self):
        print("This is a student")
```

```

class Teacher:
    def describe(self):
        print("This is a teacher")

people = [Student(), Teacher()]

for person in people:
    person.describe()

```

11. Class Variables

```

class Student:
    university = "UAF"

    def __init__(self, name):
        self.name = name

student1 = Student("Ahmad")
student2 = Student("Ali")

print(student1.university)
print(student2.university)

```

12. Class Methods

```

class Student:
    university = "UAF"

    @classmethod
    def show_university(cls):
        print(cls.university)

Student.show_university()

```

13. Static Methods

```

class DataTools:
    @staticmethod
    def valid_cgpa(cgpa):
        return 0 <= cgpa <= 4

print(DataTools.valid_cgpa(3.23))
print(DataTools.valid_cgpa(4.5))

```

14. Composition

Composition means one object uses another object. It is useful for building modular Data Science systems.

```

class DataLoader:
    def load(self):
        return [80, 75, 90, 85]

class DataAnalyzer:
    def __init__(self, loader):
        self.loader = loader

    def average(self):
        data = self.loader.load()
        return sum(data) / len(data)

analyzer = DataAnalyzer(DataLoader())
print(analyzer.average())

```

15. OOP + Exception Handling

```

class Student:
    def __init__(self, name, cgpa):
        if not 0 <= cgpa <= 4:
            raise ValueError("CGPA must be between 0 and 4")
        self.name = name
        self.cgpa = cgpa

try:
    student = Student("Ahmad", 3.23)
    print(student.name, student.cgpa)
except ValueError as error:
    print("Error:", error)

```

16. Data Science OOP Example

```
class StudentAnalyzer:
    def __init__(self, students):
        if not students:
            raise ValueError("Student list cannot be empty")

        self.students = students

        for student in students:
            if not 0 <= student["cgpa"] <= 4:
                raise ValueError(
                    f"Invalid CGPA for {student['name']}"
                )

    def average_marks(self):
        marks = [s["marks"] for s in self.students]
        return sum(marks) / len(marks)

    def highest_cgpa(self):
        return max(s["cgpa"] for s in self.students)

    def high_cgpa_students(self, threshold=3.0):
        return [s for s in self.students if s["cgpa"] >= threshold]

    def report(self):
        print("Students:", len(self.students))
        print("Average Marks:", self.average_marks())
        print("Highest CGPA:", self.highest_cgpa())
        print("CGPA >= 3.0:")
        for s in self.high_cgpa_students():
            print(s["name"], "->", s["cgpa"])

students = [
    {"name": "Ahmad", "marks": 85, "cgpa": 3.23},
    {"name": "Ali", "marks": 72, "cgpa": 3.10},
    {"name": "Usman", "marks": 65, "cgpa": 2.90},
    {"name": "Hamza", "marks": 91, "cgpa": 3.60}
]

try:
    StudentAnalyzer(students).report()
except ValueError as error:
    print("Error:", error)
```

17. Practice Tasks

1. Create a Student class with name, age, and CGPA. Create two objects.
2. Create a Calculator class with add, subtract, multiply, and divide.
3. Create a Student class with an average_marks method.
4. Create a BankAccount class with deposit and withdraw validation.
5. Create Person and Student classes using inheritance.
6. Create Shape and Rectangle classes and override area().
7. Create DataTools with a static CGPA validation method.
8. Create a Student class with a shared university class variable.
9. Create DataLoader and DataAnalyzer using composition.
10. Build a complete OOP Student Data Analyzer with validation, calculations, filtering, and a report.

18. Full Solutions

Task 1 Solution

```
class Student:
    def __init__(self, name, age, cgpa):
        self.name = name
        self.age = age
        self.cgpa = cgpa

    def display(self):
```

```

        print(self.name, self.age, self.cgpa)

Student("Ahmad", 22, 3.23).display()
Student("Ali", 21, 3.10).display()

```

Task 2 Solution

```

class Calculator:
    def add(self, a, b): return a + b
    def subtract(self, a, b): return a - b
    def multiply(self, a, b): return a * b
    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero")
        return a / b

c = Calculator()
print(c.add(10, 5))
print(c.subtract(10, 5))
print(c.multiply(10, 5))
print(c.divide(10, 5))

```

Task 3 Solution

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average(self):
        return sum(self.marks) / len(self.marks)

print(Student("Ahmad", [80, 75, 90]).average())

```

Task 4 Solution

```

class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0: raise ValueError("Deposit must be positive")
        self.balance += amount

    def withdraw(self, amount):
        if amount <= 0: raise ValueError("Withdrawal must be positive")
        if amount > self.balance: raise ValueError("Insufficient balance")
        self.balance -= amount

a = BankAccount(1000)
a.deposit(500)
a.withdraw(200)
print(a.balance)

```

Task 5 Solution

```

class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print(self.name)

class Student(Person):
    def study(self):
        print(self.name, "is studying")

s = Student("Ahmad")
s.show()
s.study()

```

Task 6 Solution

```

class Shape:
    def area(self):
        return 0

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

```

```

    def area(self):
        return self.length * self.width

print(Rectangle(10, 5).area())

```

Task 7 Solution

```

class DataTools:
    @staticmethod
    def valid_cgpa(cgpa):
        return 0 <= cgpa <= 4

print(DataTools.valid_cgpa(3.23))
print(DataTools.valid_cgpa(4.5))

```

Task 8 Solution

```

class Student:
    university = "UAF"

    def __init__(self, name):
        self.name = name

a = Student("Ahmad")
b = Student("Ali")
print(a.name, a.university)
print(b.name, b.university)

```

Task 9 Solution

```

class DataLoader:
    def load(self):
        return [80, 75, 90, 85]

class DataAnalyzer:
    def __init__(self, loader):
        self.loader = loader

    def average(self):
        data = self.loader.load()
        return sum(data) / len(data)

print(DataAnalyzer(DataLoader()).average())

```

Task 10 Solution

```

class StudentAnalyzer:
    def __init__(self, students):
        if not students:
            raise ValueError("No student data")
        self.students = students
        for s in students:
            if not 0 <= s["cgpa"] <= 4:
                raise ValueError(f"Invalid CGPA: {s['name']}")

    def average_marks(self):
        return sum(s["marks"] for s in self.students) / len(self.students)

    def highest_cgpa(self):
        return max(s["cgpa"] for s in self.students)

    def filter_students(self, threshold=3.0):
        return [s for s in self.students if s["cgpa"] >= threshold]

    def report(self):
        print("Students:", len(self.students))
        print("Average Marks:", self.average_marks())
        print("Highest CGPA:", self.highest_cgpa())
        for s in self.filter_students():
            print(s["name"], s["cgpa"])

students = [
    {"name": "Ahmad", "marks": 85, "cgpa": 3.23},
    {"name": "Ali", "marks": 72, "cgpa": 3.10},
    {"name": "Usman", "marks": 65, "cgpa": 2.90},
    {"name": "Hamza", "marks": 91, "cgpa": 3.60}
]

try:
    StudentAnalyzer(students).report()
except ValueError as e:

```

```
print("Error:", e)
```

19. Common Mistakes

1. Forgetting self in instance methods.
2. Confusing a class with an object.
3. Using a class variable when each object needs its own value.
4. Skipping validation when changing object data.
5. Creating one giant class instead of separating responsibilities.
6. Using inheritance where composition would be clearer.
7. Forgetting that `_name` is a convention, not true private access control.

20. Quick Revision

Concept	Meaning
Class	Blueprint
Object	Instance of a class
Attribute	Object data
Method	Function inside a class
<code>__init__</code>	Initializes an object
self	Current object
Inheritance	Reuse/extend parent
<code>super()</code>	Call parent functionality
Polymorphism	Same interface, different behavior
Encapsulation	Keep data and behavior together
Class method	Uses cls and class data
Static method	Utility method without self/cls
Composition	Objects built using other objects

21. Data Science Connection

OOP becomes useful as projects grow. Data loaders, cleaning components, analyzers, visualizers, model trainers, and pipelines can be organized into reusable classes. Later, many Data Science libraries will expose classes and objects, so these concepts will help you understand their APIs.

Next progression: Python → NumPy → Pandas → Data Cleaning → Data Visualization → Statistics → Machine Learning → Deep Learning.

22. Final Checklist

- ✓ I understand classes and objects.
- ✓ I can use `__init__` and `self`.
- ✓ I can create methods.
- ✓ I understand encapsulation.
- ✓ I can use inheritance and `super()`.
- ✓ I understand overriding and polymorphism.
- ✓ I know class and static methods.
- ✓ I understand composition.

- ✓ I can build an OOP-based Data Science component.
- ✓ I completed 10 practice tasks and the mini project.